Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатика и управление

КАФЕДРА ИУК4 Программное обеспечение ЭВМ, информационные технологии

Домашняя работа №2

«Сравнение скорости выполнения вычислений в браузере с помощью JS и WASM»

по дисциплине: «Фреймворки web-приложений»

Выполнил: студент группы ИУК4-82Б _____ (Подпись) Моряков В.Ю. (И.О. Фамилия)

Проверил: _____ (Подпись) Голубев И.С. (И.О. Фамилия)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

Калуга, 2026

Задание
Реализовать 2 решения задачи в браузере по варинату: на JS и WASM. Сгенерировать случайные наборы входных данных большого размера (от 10тыс.) и сравнить время выполнения двух решений.

Требования к реализации

- Можно использовать любой ЯП, компилируемый в WASM (предпочтительно Rust)
- Можно использовать любые JS-библиотеки и фреймворки для отображения данных
- (но не для выполнения вычислений!)

Вариант: Компания, занимающаяся логистикой.

Листинг программы:

```
lib.rs

mod pathfinding;
mod visualization;

use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub fn shortest_path_wasm(
    node_count: usize,
    from: Vec<u32>,
    to: Vec<u32>,
    weights: Vec<f64>,
    start: usize,
    target: usize,
) -> Result<JsValue, JsValue> {
    pathfinding::shortest_path(node_count, from, to, weights, start, target)
}

#[wasm_bindgen]
pub fn prepare_graph_visualization_wasm(
    node_count: usize,
    from: Vec<u32>,
    to: Vec<u32>,
    start: usize,
    target: usize,
    path_nodes: Vec<u32>,
    max_nodes: usize,
) -> Result<JsValue, JsValue> {
    visualization::prepare_graph_visualization(
        node_count,
        from,
        to,
        start,
        target,
        path_nodes,
        max_nodes,
    )
}

pathFinding.rs

use serde::Serialize;
use std::cmp::Ordering;
use std::collections::BinaryHeap;
```

```
use wasm_bindgen::prelude::*;

#[derive(Copy, Clone)]
struct State {
    cost: f64,
    node: usize,
}

impl Eq for State {}

impl PartialEq for State {
    fn eq(&self, other: &Self) -> bool {
        self.node == other.node && self.cost == other.cost
    }
}

impl Ord for State {
    fn cmp(&self, other: &Self) -> Ordering {
        other
            .cost
            .partial_cmp(&self.cost)
            .unwrap_or(Ordering::Equal)
            .then_with(|| other.node.cmp(&self.node))
    }
}

impl PartialOrd for State {
    fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
        Some(self.cmp(other))
    }
}

#[derive(Serialize)]
struct PathResult {
    found: bool,
    distance: Option<f64>,
    path_length: usize,
    path_nodes: Vec<u32>,
    visited_count: usize,
}

fn build_csr(
    node_count: usize,
    from: &[u32],
    to: &[u32],
    weights: &[f64],
) -> Result<(Vec<usize>, Vec<u32>, Vec<f64>), JsValue> {
    if from.len() != to.len() || to.len() != weights.len() {
        return Err(JsValue::from_str(
            "Edge arrays must have the same length.",
        ));
    }

    let edge_count = from.len();
    let mut offsets = vec![0usize; node_count + 1];

    for &source in from {
        let index = source as usize;
        if index >= node_count {
            return Err(JsValue::from_str("Source vertex is out of range."));
        }
        offsets[index + 1] += 1;
```

```
    }

    for index in 1..offsets.len() {
        offsets[index] += offsets[index - 1];
    }

    let mut cursor = offsets[..node_count].to_vec();
    let mut neighbors = vec![0u32; edge_count];
    let mut costs = vec![0.0f64; edge_count];

    for edge_index in 0..edge_count {
        let source = from[edge_index] as usize;
        let target = to[edge_index] as usize;

        if target >= node_count {
            return Err(JsValue::from_str("Target vertex is out of range."));
        }

        let slot = cursor[source];
        neighbors[slot] = to[edge_index];
        costs[slot] = weights[edge_index];
        cursor[source] += 1;
    }

    Ok((offsets, neighbors, costs))
}

fn reconstruct_path_length(previous: &[i32], start: usize, target: usize) -> usize {
    if target != start && previous[target] == -1 {
        return 0;
    }

    let mut length = 1usize;
    let mut current = target;

    while current != start {
        let parent = previous[current];
        if parent < 0 {
            return 0;
        }

        current = parent as usize;
        length += 1;
    }

    length
}

fn reconstruct_path_nodes(previous: &[i32], start: usize, target: usize) -> Vec<u32>
{
    if target != start && previous[target] == -1 {
        return Vec::new();
    }

    let mut reversed_path = Vec::new();
    let mut current = target;

    loop {
        reversed_path.push(current as u32);
        if current == start {
            break;
        }
```

```
        let parent = previous[current];
        if parent < 0 {
            return Vec::new();
        }

        current = parent as usize;
    }

    reversed_path.reverse();
    reversed_path
}

pub fn shortest_path(
    node_count: usize,
    from: Vec<u32>,
    to: Vec<u32>,
    weights: Vec<f64>,
    start: usize,
    target: usize,
) -> Result<JsValue, JsValue> {
    if start >= node_count || target >= node_count {
        return Err(JsValue::from_str("Start or target vertex is out of range."));
    }

    let (offsets, neighbors, costs) = build_csr(node_count, &from, &to, &weights)?;
    let mut distances = vec![f64::INFINITY; node_count];
    let mut previous = vec![-1i32; node_count];
    let mut visited = vec![false; node_count];
    let mut heap = BinaryHeap::new();
    let mut visited_count = 0usize;

    distances[start] = 0.0;
    heap.push(State {
        cost: 0.0,
        node: start,
    });

    while let Some(State { cost, node }) = heap.pop() {
        if visited[node] {
            continue;
        }

        visited[node] = true;
        visited_count += 1;

        if node == target {
            break;
        }

        if cost > distances[node] {
            continue;
        }

        for edge_index in offsets[node]..offsets[node + 1] {
            let next_node = neighbors[edge_index] as usize;
            let next_cost = cost + costs[edge_index];

            if next_cost < distances[next_node] {
                distances[next_node] = next_cost;
                previous[next_node] = node as i32;
                heap.push(State {
```

```
                    cost: next_cost,
                    node: next_node,
                });
            }
        }
    }

    let found = distances[target].is_finite();
    let result = PathResult {
        found,
        distance: found.then_some(distances[target]),
        path_length: if found {
            reconstruct_path_length(&previous, start, target)
        } else {
            0
        },
        path_nodes: if found {
            reconstruct_path_nodes(&previous, start, target)
        } else {
            Vec::new()
        },
        visited_count,
    };

    serde_wasm_bindgen::to_value(&result)
                .map_err(|error|  JsValue::from_str(&format!("Serialization  failed:
{error}")))
}
```

visualisation.rs

```
use serde::Serialize;
use std::collections::VecDeque;
use wasm_bindgen::prelude::*;

#[derive(Serialize)]
struct VisualizationResult {
    node_ids: Vec<u32>,
    x: Vec<f64>,
    y: Vec<f64>,
    edge_from: Vec<u32>,
    edge_to: Vec<u32>,
    path_indices: Vec<u32>,
    start_index: usize,
    target_index: usize,
}

fn build_adjacency(
    node_count: usize,
    from: &[u32],
    to: &[u32],
) -> Result<Vec<Vec<usize>>, JsValue> {
    if from.len() != to.len() {
        return Err(JsValue::from_str(
            "Edge arrays for visualization must have the same length.",
        ));
    }

    let mut adjacency = vec![Vec::new(); node_count];

    for index in 0..from.len() {
        let source = from[index] as usize;
```

```
        let target = to[index] as usize;

        if source >= node_count || target >= node_count {
            return Err(JsValue::from_str(
                "Visualization edge contains a vertex out of range.",
            ));
        }

        adjacency[source].push(target);
    }

    for neighbors in &mut adjacency {
        neighbors.sort_unstable();
        neighbors.dedup();
    }

    Ok(adjacency)
}

fn collect_sampled_nodes(
    adjacency: &[Vec<usize>],
    start: usize,
    target: usize,
    path_nodes: &[u32],
    max_nodes: usize,
) -> Vec<usize> {
    let node_count = adjacency.len();
    let limit = max_nodes.max(2).min(node_count);
    let mut selected = Vec::with_capacity(limit);
    let mut used = vec![false; node_count];
    let mut queue_start = VecDeque::new();
    let mut queue_target = VecDeque::new();

    for &node in path_nodes {
        let node_index = node as usize;
        if node_index < node_count && !used[node_index] && selected.len() < limit {
            used[node_index] = true;
            selected.push(node_index);
        }
    }

    if !used[start] && selected.len() < limit {
        used[start] = true;
        selected.push(start);
    }
    queue_start.push_back(start);

    if !used[target] && selected.len() < limit {
        used[target] = true;
        selected.push(target);
    }
    queue_target.push_back(target);

            while   selected.len()   <   limit   &&   (!queue_start.is_empty()   ||   !
queue_target.is_empty()) {
        let mut progressed = false;

        if let Some(node) = queue_start.pop_front() {
            for &neighbor in &adjacency[node] {
                if selected.len() >= limit {
                    break;
                }
```

```
                if !used[neighbor] {
                    used[neighbor] = true;
                    selected.push(neighbor);
                    queue_start.push_back(neighbor);
                    progressed = true;
                }
            }
        }

        if selected.len() >= limit {
            break;
        }

        if let Some(node) = queue_target.pop_front() {
            for &neighbor in &adjacency[node] {
                if selected.len() >= limit {
                    break;
                }

                if !used[neighbor] {
                    used[neighbor] = true;
                    selected.push(neighbor);
                    queue_target.push_back(neighbor);
                    progressed = true;
                }
            }
        }

        if !progressed && queue_start.is_empty() && queue_target.is_empty() {
            break;
        }
    }

    selected.sort_unstable();
    selected
}

fn build_sampled_edges(
    adjacency: &[Vec<usize>],
    selected_nodes: &[usize],
) -> (Vec<u32>, Vec<u32>) {
    let mut index_by_node = vec![usize::MAX; adjacency.len()];
    for (sample_index, &node) in selected_nodes.iter().enumerate() {
        index_by_node[node] = sample_index;
    }

    let mut edge_from = Vec::new();
    let mut edge_to = Vec::new();

    for (sample_index, &node) in selected_nodes.iter().enumerate() {
        for &neighbor in &adjacency[node] {
            let mapped_neighbor = index_by_node[neighbor];
            if mapped_neighbor != usize::MAX && node < neighbor {
                edge_from.push(sample_index as u32);
                edge_to.push(mapped_neighbor as u32);
            }
        }
    }

    (edge_from, edge_to)
}
```

```
fn build_sampled_adjacency(node_count: usize, edge_from: &[u32], edge_to: &[u32]) ->
Vec<Vec<usize>> {
    let mut adjacency = vec![Vec::new(); node_count];

    for edge_index in 0..edge_from.len() {
        let source = edge_from[edge_index] as usize;
        let target = edge_to[edge_index] as usize;

        adjacency[source].push(target);
        adjacency[target].push(source);
    }

    adjacency
}

fn bfs_distances(adjacency: &[Vec<usize>], source: usize) -> Vec<usize> {
    let mut distances = vec![usize::MAX; adjacency.len()];
    let mut queue = VecDeque::new();

    distances[source] = 0;
    queue.push_back(source);

    while let Some(node) = queue.pop_front() {
        let next_distance = distances[node] + 1;
        for &neighbor in &adjacency[node] {
            if distances[neighbor] == usize::MAX {
                distances[neighbor] = next_distance;
                queue.push_back(neighbor);
            }
        }
    }

    distances
}

fn stable_jitter(node_id: u32, salt: u32) -> f64 {
    let mixed = node_id
        .wrapping_mul(1_664_525)
        .wrapping_add(1_013_904_223)
        .wrapping_add(salt.wrapping_mul(374_761_393));
    (mixed % 10_000) as f64 / 10_000.0
}

fn compute_layered_layout(
    node_count: usize,
    node_ids: &[u32],
    edge_from: &[u32],
    edge_to: &[u32],
    start_index: usize,
    target_index: usize,
    path_indices: &[u32],
) -> (Vec<f64>, Vec<f64>) {
    if node_count == 0 {
        return (Vec::new(), Vec::new());
    }

    let adjacency = build_sampled_adjacency(node_count, edge_from, edge_to);
    let distance_from_start = bfs_distances(&adjacency, start_index);
    let distance_from_target = bfs_distances(&adjacency, target_index);
    let mut columns: Vec<Vec<usize>> = vec![Vec::new(); 12];
    let mut path_position = vec![usize::MAX; node_count];
```

```
    for (index, &node_index) in path_indices.iter().enumerate() {
        path_position[node_index as usize] = index;
    }

    for node_index in 0..node_count {
        let start_distance = distance_from_start[node_index];
        let target_distance = distance_from_target[node_index];
        let column = if node_index == start_index {
            0
        } else if node_index == target_index {
            columns.len() - 1
        } else if path_position[node_index] != usize::MAX && path_indices.len() > 1 {
            ((path_position[node_index] as f64 / (path_indices.len() - 1) as f64)
                * (columns.len() - 1) as f64)
                .round() as usize
        } else if start_distance != usize::MAX && target_distance != usize::MAX {
                        let  ratio  =  start_distance  as  f64  /  (start_distance  +
target_distance).max(1) as f64;
            (ratio * (columns.len() - 1) as f64).round() as usize
        } else if start_distance != usize::MAX {
            ((start_distance.min(columns.len() - 2)) + 1).min(columns.len() - 2)
        } else if target_distance != usize::MAX {
                (columns.len()  -  2).saturating_sub(target_distance.min(columns.len()  -
2))
        } else {
            node_index % columns.len()
        };

        columns[column].push(node_index);
    }

    for bucket in &mut columns {
        bucket.sort_by(|&left, &right| {
            path_position[left]
                .cmp(&path_position[right])
                                                                                .then_with(||
distance_from_start[left].cmp(&distance_from_start[right]))
                                                                                .then_with(||
distance_from_target[left].cmp(&distance_from_target[right]))
                .then_with(|| adjacency[right].len().cmp(&adjacency[left].len()))
                .then_with(|| node_ids[left].cmp(&node_ids[right]))
        });
    }

    let mut x = vec![0.0f64; node_count];
    let mut y = vec![0.0f64; node_count];
    let column_count = columns.len();

    for (column_index, bucket) in columns.iter().enumerate() {
        if bucket.is_empty() {
            continue;
        }

        let base_x = if column_count == 1 {
            0.5
        } else {
            0.08 + column_index as f64 / (column_count - 1) as f64 * 0.84
        };

        for (row_index, &node_index) in bucket.iter().enumerate() {
```

```
                    let base_y = if path_position[node_index] != usize::MAX &&
path_indices.len() > 1 {
                let ratio = path_position[node_index] as f64 / (path_indices.len() - 1)
as f64;
                0.16 + ratio * 0.68
            } else if bucket.len() == 1 {
                0.5
            } else {
                0.1 + row_index as f64 / (bucket.len() - 1) as f64 * 0.8
            };
            let jitter_x = (stable_jitter(node_ids[node_index], 17) - 0.5) * 0.022;
            let jitter_y = (stable_jitter(node_ids[node_index], 43) - 0.5) * 0.03;

            x[node_index] = (base_x + jitter_x).clamp(0.06, 0.94);
            y[node_index] = (base_y + jitter_y).clamp(0.08, 0.92);
        }
    }

    x[start_index] = 0.08;
    y[start_index] = 0.16;
    x[target_index] = 0.92;
    y[target_index] = 0.84;

    (x, y)
}

pub fn prepare_graph_visualization(
    node_count: usize,
    from: Vec<u32>,
    to: Vec<u32>,
    start: usize,
    target: usize,
    path_nodes: Vec<u32>,
    max_nodes: usize,
) -> Result<JsValue, JsValue> {
    if node_count == 0 {
        return Err(JsValue::from_str("Graph is empty."));
    }

    if start >= node_count || target >= node_count {
        return Err(JsValue::from_str(
            "Start or target vertex is out of range for visualization.",
        ));
    }

    let adjacency = build_adjacency(node_count, &from, &to)?;
    let selected_nodes = collect_sampled_nodes(&adjacency, start, target, &path_nodes,
max_nodes);
    let start_index = selected_nodes
        .iter()
        .position(|&node| node == start)
        .ok_or_else(|| JsValue::from_str("Start node is missing in visualization."))?;
    let target_index = selected_nodes
        .iter()
        .position(|&node| node == target)
                    .ok_or_else(|| JsValue::from_str("Target node is missing in
visualization."))?;
    let (edge_from, edge_to) = build_sampled_edges(&adjacency, &selected_nodes);
          let node_ids: Vec<u32> = selected_nodes.iter().map(|&node| node as
u32).collect();

    let mut path_indices = Vec::new();
```

```
    for &node in &path_nodes {
        if let Some(index) = selected_nodes
            .iter()
            .position(|&selected_node| selected_node == node as usize)
        {
            path_indices.push(index as u32);
        }
    }

    let (x, y) = compute_layered_layout(
        selected_nodes.len(),
        &node_ids,
        &edge_from,
        &edge_to,
        start_index,
        target_index,
        &path_indices,
    );

    let result = VisualizationResult {
        node_ids,
        x,
        y,
        edge_from,
        edge_to,
        path_indices,
        start_index,
        target_index,
    };

    serde_wasm_bindgen::to_value(&result).map_err(|error| {
        JsValue::from_str(&format!(
            "Visualization serialization failed: {error}"
        ))
    })
}
```

Страницы:

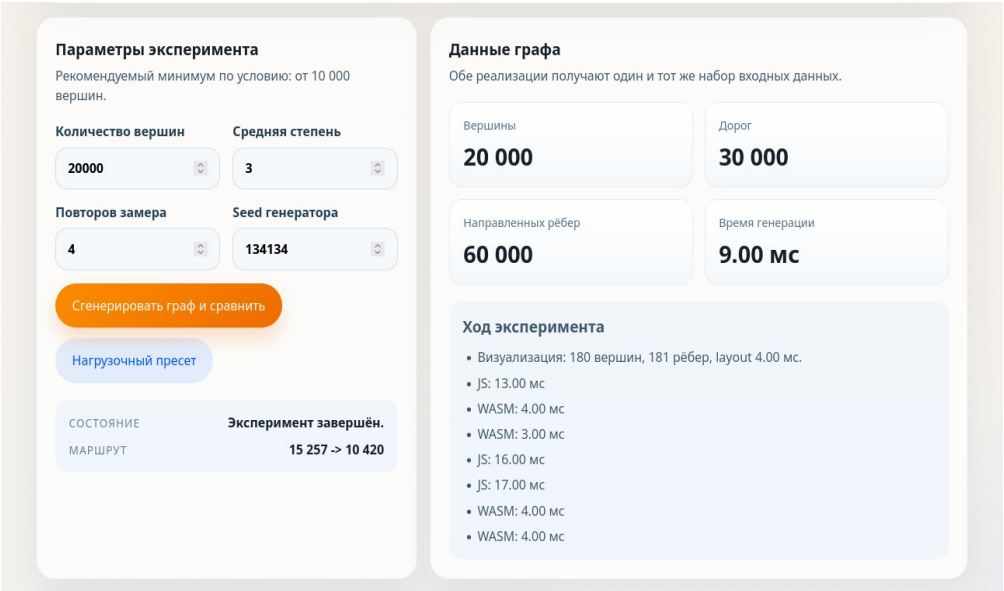

Рисунок 1 Главная страница

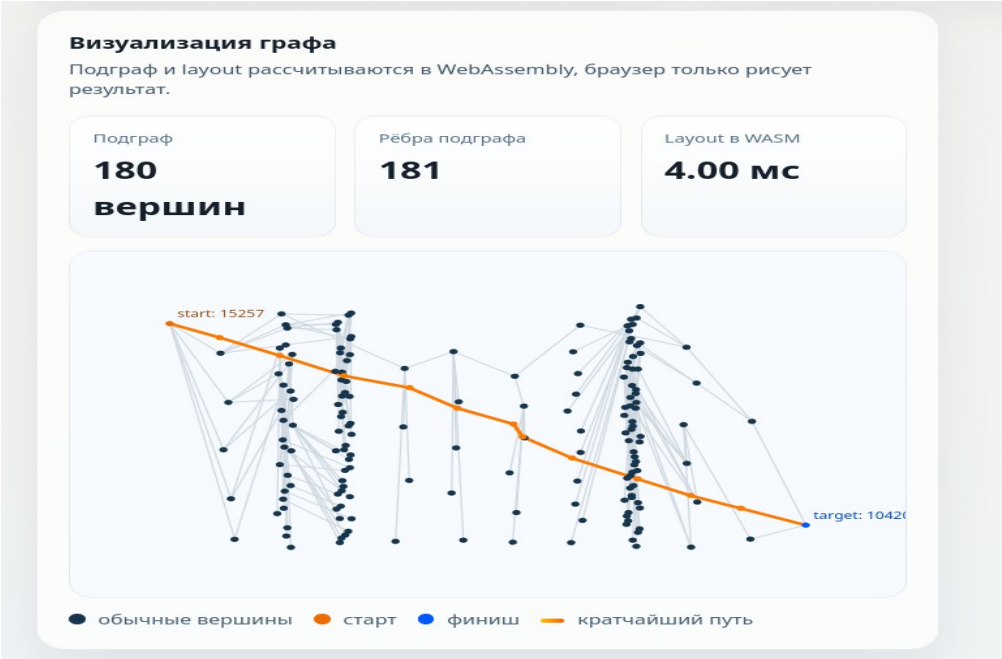

Рисунок 2 Визуализация графа

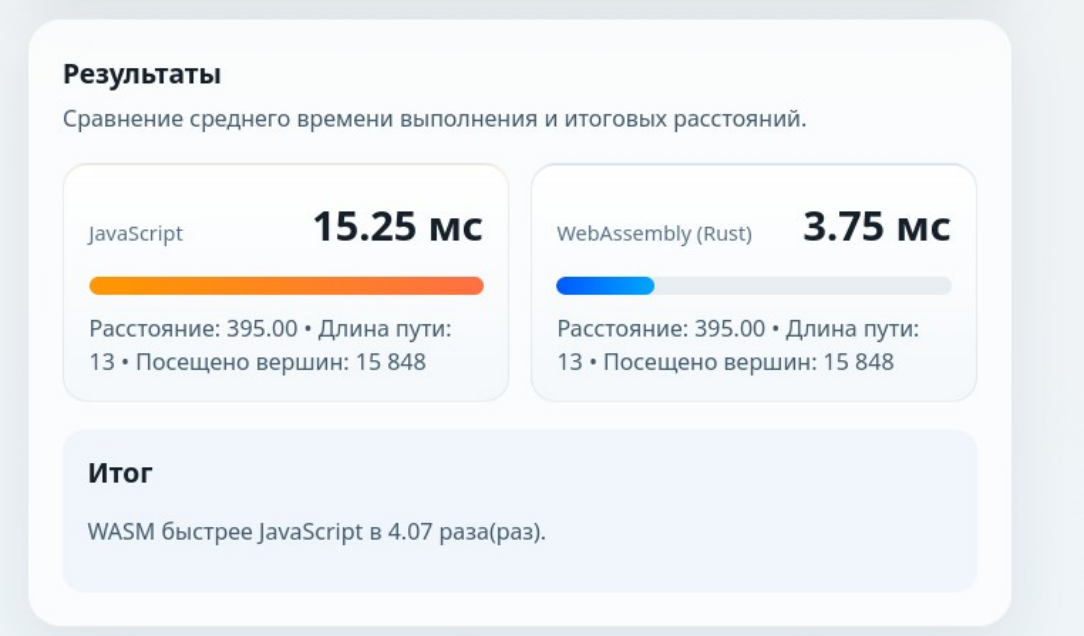

Рисунок 3 Сравнение результата nativejs и wasm

Вывод: в ходе работы были реализованы два решения задачи в браузере: на JavaScript и на WASM. Для сравнения производительности были сгенерированы большие наборы входных данных размером от 10 000 элементов. Тестирование показало, что обе реализации корректно выполняют поставленную задачу, но отличаются по скорости работы. JavaScript проще в разработке и отладке, а WASM может быть эффективнее при больших вычислительных нагрузках. Таким образом, выбор между JavaScript и WASM зависит от требований к производительности и удобству реализации.